RISC-V C API Specification

Authors: several contributors

Version v0.0.0, 2025-07-17: This document is in Development state. Change should be expected.

Table of Contents

Preamble . . . 1
List of tables . . . 2
Copyright and License Information . . . 3
Scope . . . 4
Contributors . . . 5
1. Preprocessor Definitions . . . 6
1.1. __riscv_v_min_vlen . . . 6
1.2. __riscv_v_elen . . . 7
1.3. __riscv_v_elen_fp . . . 7
1.4. __riscv_misaligned_{fast,slow,avoid} . . . 7
1.5. Architecture Extension Test Macros . . . 8
1.6. ABI Related Preprocessor Definitions . . . 8
1.7. Code Model Related Preprocessor Definitions . . . 8
1.8. Deprecated Preprocessor Definitions . . . 9
2. Function Attributes . . . 10
2.1. `__attribute__((naked))` . . . 10
2.2. `__attribute__((interrupt))`, `__attribute__((interrupt("supervisor")))`, `__attribute__((interrupt("machine")))` . . . 10
2.3. `__attribute__((target("<ATTR-STRING>")))` . . . 10
2.4. `__attribute__((target_clones("<TARGET-CLONES-ATTR-STRING>", ...)))` . . . 12
2.5. `__attribute__((target_version("<TARGET-VERSION-ATTR-STRING>")))` . . . 13
2.6. riscv_vector_cc . . . 14
3. Intrinsic Functions . . . 16
3.1. NTLH Intrinsics . . . 16
3.2. Prefetch Intrinsics . . . 17
3.3. Scalar Bit Manipulation Extension Intrinsics . . . 18
3.4. Scalar Cryptography Extension Intrinsics . . . 20
3.5. May-Be-Operations Extension Intrinsics . . . 21
4. Constraints on Operands of Inline Assembly Statements . . . 23
4.1. The Difference Between `m` and `A` Constraints . . . 24
4.2. Operand Modifiers . . . 24
5. Function Multi-version . . . 26
5.1. Extension Bitmask . . . 26
5.1.1. Extension Bitmask Definitions . . . 27
5.2. Version Selection . . . 29

Preamble

This document is in the Development state

Assume everything can change. This draft specification will change before being accepted as standard, so implementations made to this draft specification will likely not conform to the future standard.

List of tables

Table 1. Preprocessor Definitions
Table 2. ABI Related Preprocessor Definitions
Table 3. Code Model Related Preprocessor Definitions
Table 4. Deprecated Preprocessor Definitions
Table 5. Domain Value to Instruction Mapping
Table 6. Prefetch Functions to Assembly Mapping
Table 7. Scalar Bit Manipulation Extension Intrinsics
Table 8. Scalar Cryptography Extension Intrinsics
Table 9. May-Be-Operations Extension Intrinsics
Table 10. Constraints on Operands of Inline Assembly Statements
Table 11. Operand Modifiers

Copyright and License Information

The C API Sepecification is © 2018 Palmer Dabbelt palmer@sifive.com and © 2018 SiFive, Inc.

It is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0).

The full license text is available at creativecommons.org/licenses/by/4.0/

Scope

This file contains the RISC-V additions to the standard C APIs. It merges together the compiler command-line API, the preprocessor API, and the C language extensions (function attributes and intrinsic functions).

Contributors

See github.com/riscv-non-isa/riscv-c-api-doc/graphs/contributors.

Chapter 1. Preprocessor Definitions

Table 1. Preprocessor Definitions

Mame	Value	When defined
`__riscv`	1	Always defined.
`__riscv_xlen`	• 32 for rv32 • 64 for rv64 • 128 for rv128	Always defined.
`__riscv_flen`	• 32 if the F extension is available **or** • 64 if `D` extension available **or** • 128 if `Q` extension available	`F` extension is available.
`__riscv_32e`	1	RV32E is available.
`__riscv_64e`	1	RV64E is available.
`__riscv_vector`	1	Implies that any of the vector extensions (`v` or `zve*`) is available
`__riscv_v_min_vlen`	<N> (see `__riscv_v_min_vlen`)	The `V` extension or one of the `Zve*` extensions is available.
`__riscv_v_elen`	<N> (see `__riscv_v_elen`)	The `V` extension or one of the `Zve*` extensions is available.
`__riscv_v_elen_fp`	<N> (see `__riscv_v_elen_fp`)	The `V` extension or one of the `Zve*` extensions is available.
`__riscv_misaligned_fast`	1	Scalar misaligned accesses are fast.
`__riscv_misaligned_slow`	1	Scalar misaligned accesses are supported, but may be substantially slower than aligned accesses.
`__riscv_misaligned_avoid`	1	Scalar misaligned accesses are not supported and could trap. (see `__riscv_misaligned_{fast,slow,avoid}`)

1.1. __riscv_v_min_vlen

The `__riscv_v_min_vlen` macro expands to the minimal VLEN, in bits, mandated by the available vector extension, if any.

The value of `__riscv_v_min_vlen` is defined by the following rules:

- 128, if the `V` extension is present;
- 32, if one of the `Zve32{x,f}` extensions is present;
- 64, if one of the `Zve64{x,f,d}` extensions is present;
- `N`, if one of the `Zvl<N>b` extensions, `N` in `{32,64,128,256,512,1024}`, is present.

If multiple rules apply, the maximum value is taken. If none of the rules apply, `__riscv_v_min_vlen` is undefined.

Examples:

- `__riscv_v_min_vlen` is 128 for `rv64gcv`
- `__riscv_v_min_vlen` is 512 for `rv32gcv_zvl512b`
- `__riscv_v_min_vlen` is 256 for `rv32gcv_zvl32b_zvl256b`
- `__riscv_v_min_vlen` is 128 for `rv64gcv_zvl32b`

1.2. __riscv_v_elen

The `__riscv_v_elen` macro expands to the supported element length, in bits, of any non-floating-point vector operand of any vector instruction in the available vector extension, if any. (Stricter upper bounds may apply to particular operands of particular instructions.)

The value of `__riscv_v_elen` is defined by the following rules:

- 64, if the `V` extension or one of the `Zve64{x,f,d}` extensions is present; and
- 32, if one of the `Zve32{x,f}` extensions is present. If multiple rules apply, the maximum value is taken. If none of the rules apply, `__riscv_v_elen` is undefined.

1.3. __riscv_v_elen_fp

The `__riscv_v_elen_fp` macro expands to the supported element length, in bits, of any floating-point vector operand of any vector instruction in the available vector extension, if any. (Stricter upper bounds may apply to particular operands of particular instructions.)

The value of `__riscv_v_elen_fp` is defined by the following rules:

- 64, if one of the `V` or `Zve64d` extensions is present;
- 32, if one of the `Zve{32,64}f` extensions is present; and
- 0, if one of the `Zve{32,64}x` extensions is present. If multiple rules apply, the maximum value is taken. If none of the rules apply, `__riscv_v_elen_fp` is undefined.

1.4. __riscv_misaligned_{fast,slow,avoid}

These can be used in common library code to compile time segregate code which relies on scalar misaligned access being fast or not. A typical compiler could (but not necessarily) map fast variant to -mno-strict-align and avoid to -mstrict-align, if specified. Perhaps obvious, but these are mutually

exclusive, so only one is defined at a time for a compilation unit.

1.5. Architecture Extension Test Macros

Architecture extension test macros allow checking the availability and version of extensions at compile-time. These feature macros are optional, and compilers that support them provide a `__riscv_arch_test` macro (with the value `1`).

The naming rule for the test macros is `__riscv_<ext_name>`, where `<ext_name>` is all lower-case. Examples:

- The test macro for the `A` extension is `__riscv_a`.
- The test macro for the `Zifencei` extension is `__riscv_zifencei`.
- The test macro for the `XVentanaCondOps` extension is `__riscv_xventantacondops`.

Besides extensions, test macros also exist for ISA bases following the same pattern. Examples:

- The test macro for the `I` base is `__riscv_i`.
- The test macro for the `E` base is `__riscv_e`.

The value of the test macros is derived from its version using the following formula:

```
<MAJOR_VERSION> * 1,000,000 + <MINOR_VERSION> * 1,000 + <REVISION_VERSION>
```

For example:

- F-extension v2.2 will define `__riscv_f` as `2002000`.

1.6. ABI Related Preprocessor Definitions

Table 2. ABI Related Preprocessor Definitions

Name	Value	When defined
`__riscv_abi_rve`	1	Defined if using `ilp32e` or `lp64e` ABI
`__riscv_float_abi_soft`	1	Defined if using `ilp32`, `ilp32e`, `lp64` or `lp64e` ABI.
`__riscv_float_abi_single`	1	Defined if using `ilp32f` or `lp64f` ABI.
`__riscv_float_abi_double`	1	Defined if using `ilp32d` or `lp64d` ABI.
`__riscv_float_abi_quad`	1	Defined if using `ilp32q` or `lp64q` ABI.

1.7. Code Model Related Preprocessor Definitions

Table 3. Code Model Related Preprocessor Definitions

Name	Value	When defined
`__riscv_cmodel_medlow`	1	Defined if using `medlow` code model.

`__riscv_cmodel_medany`	1	Defined if using `medany` code model.
`__riscv_cmodel_large`	1	Defined if using `large` code model.

1.8. Deprecated Preprocessor Definitions

Table 4. Deprecated Preprocessor Definitions

Name	Value	When defined	Alternative
`__riscv_cmodel_pic`	1	GCC defines this when compiling with `-fPIC`, `-fpic`, `-fPIE` or `-fpie`.	`__PIC__` or `__PIE__`
`__riscv_mul`	1	`M` extension is available.	`__riscv_m`
`__riscv_div`	1	`M` extension is available and `-mno-div` is not given.{fn-1}	`__riscv_m`
`__riscv_muldiv`	1	`M` extension is available and `-mno-div` is not given.{fn-1}	`__riscv_m`
`__riscv_atomic`	1	`A` extension is available.	`__riscv_a`
`__riscv_fdiv`	1	`F` extension is available and `-mno-fdiv` is not given.{fn-1}	`__riscv_f` or `__riscv_d`
`__riscv_fsqrt`	1	`F` extension is available and `-mno-fdiv` is not given.{fn-1}	`__riscv_f` or `__riscv_d`
`__riscv_compressed`	1	`C` extension is available.	`__riscv_c`

Chapter 2. Function Attributes

2.1. `__attribute__((naked))`

The compiler won't generate the prologue/epilogue for those functions with `naked` attributes. This attribute is usually used when you want to write a function with an inline assembly body.

This attribute is incompatible with the `interrupt` attribute.

> Be aware that compilers might have further restrictions on naked functions. Please consult your compiler's manual for more information.

2.2. `__attribute__((interrupt))`, `__attribute__((interrupt("supervisor")))`, `__attribute__((interrupt("machine")))`

The interrupt attribute specifies that a function is an interrupt handler. The compiler will save/restore all used registers in the prologue/epilogue regardless of the ABI, all used registers including floating point register/vector register if `F` extension/vector extension is enabled. If F or V CSRs may be modified by an interrupt function, they must be saved by the compiler.

The interrupt attribute can have optional parameters to specify the mode. The possible values are `supervisor`, or `machine`, or vendor-specific values. The default value `machine` is used, if the mode is not specified.

The compiler should raise an error if a function declares incompatible modes, or an undefined mode. The `supervisor` mode is incompatible with the `machine` mode.

This attribute is incompatible with the `naked` attribute.

2.3. `__attribute__((target("<ATTR-STRING>")))`

The `target` attribute is used to enable a set of features or extensions for a function.

For instance, you can enable the `v` extension for a specific function even if the `-march` or `-mcpu` options do not include the `v` extension. Importantly, this won't alter the global settings. Here is an example:

```
__attribute__((target("arch=+v")))
int foo(int a)
{
  return a + 5;
}
```

Using the `target` attribute for a function should not affect the translation unit scope build attributes. For example, if a file is compiled with `-march=rv64ima` and a function is declared with

`__attribute__((target("arch=+zbb")))`, the `Tag_RISCV_arch` build attribute should remain `rv64ima`, not `rv64ima_zbb`.

The compiler may emit a mapping symbol at the beginning of a function with the target attribute if the function utilizes a different set of ISA extensions.

`<ATTR-STRING>` can specify the following target attributes:

- `arch=`: Adds extra extensions or overrides the `-march` value specified via the command line for the function.
- `tune=`: Specifies the pipeline model and cost model associated with a specific microarchitecture or core for the function.
- `cpu=`: Specifies the pipeline mode, cost model, and extension settings for the function.

The interactions among the `arch`, `tune`, and `cpu` attributes mirror those of the `-march`, `-mtune`, and `-mcpu` options. The `cpu` attribute can be seen as a combination of `arch` + `tune` but holds a lower priority than the other two. For instance, `cpu=sifive-u74` equates to `arch=rv64gc` and `tune=sifive-7-series`. However, if values for `arch=` or `tune=` are provided, they will override the `cpu` value. Therefore, `cpu=sifive-u74;arch=rv64g` is equivalent to `arch=rv64g;tune=sifive-7-series`, and `cpu=sifive-u74;tune=sifive-5-series` is equivalent to `arch=rv64gc;tune=sifive-5-series`.

The compiler should emit error if the same type of attribute is specified more than once. For example, `arch=+zbb;arch=+zba`, compiler should emit error because `arch` has specified twice.

The compiler should emit error if target attribute has specified more than once. For example, `__attribute__((target("arch=+v")))` `__attribute__((target("arch=+zbb")))` `int foo(int a)` , compiler should emit error because target attribute has specified twice.

The interactions between the attribute and the command-line option are specified below:

- `arch=`: Its behavior depends on the syntax used: 1) Adding extra extensions: It will merge the extension list with the `-march` option. 2) If a full architecture string is specified by `arch=`, it will override the `-march` option.
- `tune=`: Overrides the `-mtune` option and the pipeline model and cost model part of `-mcpu`.
- `cpu=`: Overrides the `-mcpu` option, overrides the `-mtune` option if `tune=` is not present, and overrides the `-march` option if `arch=` is not present.

The syntax of `<ATTR-STRING>` describes below:

```
ATTR-STRING := ATTR-STRING ';' ATTR
             | ATTR

ATTR        := ARCH-ATTR
             | CPU-ATTR
             | TUNE-ATTR

ARCH-ATTR   := 'arch=' EXTENSIONS-OR-FULLARCH
```

```
EXTENSIONS-OR-FULLARCH := <EXTENSIONS>
                        | <FULLARCHSTR>

EXTENSIONS             := <EXTENSION> ',' <EXTENSIONS>
                        | <EXTENSION>

FULLARCHSTR            := <full-arch-string>

EXTENSION              := <OP> <EXTENSION-NAME> <VERSION>

OP                     := '+'

VERSION                := [0-9]+ 'p' [0-9]+
                        | [1-9][0-9]*
                        |

EXTENSION-NAME         := Naming rule is defined in RISC-V ISA manual

CPU-ATTR    := 'cpu=' <valid-cpu-name>
TUNE-ATTR   := 'tune=' <valid-tune-name>
```

The target attribute does not support multi-versioning. The compiler should emit an error if a function is defined more than once. For example, the following code should trigger an error because foo is declared twice:

```
__attribute__((target("arch=+v"))) int foo(void) { return 0; }
__attribute__((target("arch=+zbb"))) int foo(void) { return 1; }
```

2.4. `__attribute__((target_clones("<TARGET-CLONES-ATTR-STRING>", ...)))`

The `target_clones` attribute is used to create multiple versions of a function. The compiler will emit multiple versions based on the provided arguments.

Each `TARGET-CLONES-ATTR-STRING` defines a distinguished version of the function. The `TARGET-CLONES-ATTR-STRING` list must include `default` indicating the translation unit scope build attributes.

The syntax of `<TARGET-CLONES-ATTR-STRING>` describes below:

```
TARGET-CLONES-ATTR-STRING := 'default'
                           | ATTR-STRINGS

ATTR-STRINGS              := ATTR-STRING
                           | ';' ATTR-STRINGS

ATTR-STRING               := ARCH-ATTR ';' PRIORITY-ATTR
                           | PRIORITY-ATTR ';' ARCH-ATTR
                           | ARCH-ATTR
```

```
ARCH-ATTR                 := 'arch=' EXTENSIONS

PRIORITY-ATTR             := 'priority=' DIGITS

DIGITS                    := [0-9]+

EXTENSIONS              := <EXTENSION> ',' <EXTENSIONS>
                         | <EXTENSION>

EXTENSION               := <OP> <EXTENSION-NAME> <VERSION>

OP                      := '+'

VERSION                 := [0-9]+ 'p' [0-9]+
                         | [1-9][0-9]*
                         |

EXTENSION-NAME          := Naming rule is defined in RISC-V ISA manual
```

For example, the following `foo` function will have three versions but share the same function signature.

```
__attribute__((target_clones("arch=+v;priority=2", "default", "arch=+zbb;priority=1
")))
int foo(int a)
{
  return a + 5;
}

int bar() {
  // foo will be resolved by ifunc
  return foo(1);
}
```

The `priority` accepts a digit as the version priority during [Version Selection](#version-selection). If `priority` isn't specified, then the priority of version defaults to zero.

It makes the compiler trigger the function multi-version, when there exist more than one version for the same function signature.

2.5. `__attribute__((target_version("<TARGET-VERSION-ATTR-STRING>")))`

The `target_version` attribute is used to create one version of a function. Functions with the same signature may exist with multiple versions in the same translation unit.

Each `TARGET-VERSION-ATTR-STRING` defines a distinguished version of the function. If there is more

than one version for the same function, it must have `default` one that indicating the translation unit scope build attributes.

The syntax of `<TARGET-VERSION-ATTR-STRING>` is the same as described above for `<TARGET-CLONES-ATTR-STRING>`.

For example, the following foo function has three versions.

```
__attribute__((target_version("arch=+v;priority=1")))
int foo(int a)
{
  return a + 5;
}

__attribute__((target_version("arch=+zbb;priority=2")))
int foo(int a)
{
  return a + 5;
}

__attribute__((target_version("default")))
int foo(int a)
{
  return a + 5;
}

int bar() {
  // foo will be resolved by ifunc
  return foo(1);
}
```

The `priority` accepts a digit as the version priority during [Version Selection](#version-selection). If `priority` isn't specified, then the priority of version defaults to zero.

The `default` version does not accept the priority.

It makes the compiler trigger the function multi-version when there exist more than one version for the same function signature.

2.6. riscv_vector_cc

Supported Syntaxes: .Supported Syntaxes:

Style	Syntax
GNU	`__attribute__((riscv_vector_cc))`
C++11	`[[riscv::vector_cc]]`
C23	`[[riscv::vector_cc]]`

Functions declared with this attribute will use to the standard vector calling convention variant as defined in the RISC-V psABI, even if the function has vector arguments or a return value.

Chapter 3. Intrinsic Functions

Intrinsic functions (or intrinsics or built-ins) are expanded into instruction sequences by compilers. They typically provide access to functionality that is otherwise not synthesizable by compilers. Some intrinsics expand to different code sequences depending on the available instructions from the enabled ISA extensions.

Compilers typically come with their own architecture-independent intrinsics (e.g. synchronization primitives, byte-swap, etc.). The RISC-V compiler backend can define additional target-specific intrinsics. Providing functionality via architecture-independent intrinsics is the preferred method, as it improves code portability.

Some intrinsics are only available if a particular header file is included. RISC-V header files that enable intrinsics require the prefix `riscv_` (e.g. `riscv_vector.h` or `riscv_crypto.h`).

RISC-V specific intrinsics use the common prefix `__riscv_` to avoid namespace collisions.

The intrinsic name describes the functional behaviour of the function. In case the functionality can be expressed with a single instruction, the instruction's name (any '.' replaced by '_') is the preferred choice. Note, that intrinsics that are restricted to RISC-V vendor extensions need to include the vendor prefix (as documented in the RISC-V toolchain conventions).

If intrinsics are available for multiple data types, then function overloading is preferred over multiple type-specific functions. In case a function is only available for one data type and this type cannot be derived from the function's name, then the type should be appended to the function name, delimited by a '_' character. Typical type postfixes are "32" (32-bit), "i32" (signed 32-bit), "i8m4" (vector register group consisting of 4 signed 8-bit vector registers).

RISC-V intrinsics follow the following naming rule:

```
INTRINSIC ::= PREFIX NAME [ '_' TYPE ]
PREFIX ::= "__riscv_"
NAME ::= Name of the intrinsic function.
TYPE ::= Optional type postfix.
```

RISC-V intrinsics examples:

```
#include <riscv_vector.h> // make RISC-V vector intrinsics available
vint8m1_t __riscv_vadd_vv_i8m1(vint8m1_t vs2, vint8m1_t vs1, size_t vl); // vadd.vv
vd, vs2, vs1
```

3.1. NTLH Intrinsics

The RISC-V zihintntl extension provides the RISC-V specific intrinsic functions for generating non-temporal memory accesses. These intrinsic functions provide the domain parameter to specify the behavior of memory accesses.

In order to access the RISC-V NTLH intrinsics, it is necessary to include the header file `riscv_ntlh.h`.

The functions are only available if the compiler enables the zihintntl extension.

```
type __riscv_ntl_load (type *ptr, int domain);
void __riscv_ntl_store (type *ptr, type val, int domain);
```

There are overloaded functions of `__riscv_ntl_load` and `__riscv_ntl_store`. When these intrinsic functions omit the `domain` argument, the `domain` is implied as `__RISCV_NTLH_ALL`.

```
type __riscv_ntl_load (type *ptr);
void __riscv_ntl_store (type *ptr, type val);
```

The types currently supported are:

- Integer types.
- Floating-point types.
- Fixed-length vector types.

The `domain` parameter could pass the following values. Each one is mapped to the specific zihintntl instruction.

```
enum {
  __RISCV_NTLH_INNERMOST_PRIVATE = 2,
  __RISCV_NTLH_ALL_PRIVATE,
  __RISCV_NTLH_INNERMOST_SHARED,
  __RISCV_NTLH_ALL
};
```

Table 5. Domain Value to Instruction Mapping

Domain Value	Instruction
`__RISCV_NTLH_INNERMOST_PRIVATE`	`ntl.p1`
`__RISCV_NTLH_ALL_PRIVATE`	`ntl.pall`
`__RISCV_NTLH_INNERMOST_SHARED`	`ntl.s1`
`__RISCV_NTLH_ALL`	`ntl.all`

3.2. Prefetch Intrinsics

The Zicbop extension provides the prefetch instruction to allow users to optimize data access patterns by providing hints to the hardware regarding future data accesses. It is supported through a compiler-defined built-in function with three arguments that specify its behavior.

```
void __builtin_prefetch(const void *addr, int rw, int locality)
```

The locality for the built-in `__builtin_prefetch` function in RISC-V can be achieved using the Non-Temporal Locality Hints (Zihintntl) extension. When a Non-Temporal Locality (NTL) Hints instruction is applied to prefetch instruction, a cache line should be prefetched into a cache level that is higher than the level specified by the NTL.

The following table presents the mapping from the `__builtin_prefetch` function to the corresponding assembly instructions assuming the presence of the Zihintntl and Zicbop extensions.

Table 6. Prefetch Functions to Assembly Mapping

Prefetch function	Assembly
`__builtin_prefetch(ptr, 0, 0 /* locality */);`	`ntl.all + prefetch.r (ptr)`
`__builtin_prefetch(ptr, 0, 1 /* locality */);`	`ntl.pall + prefetch.r (ptr)`
`__builtin_prefetch(ptr, 0, 2 /* locality */);`	`ntl.p1 + prefetch.r (ptr)`
`__builtin_prefetch(ptr, 0, 3 /* locality */);`	`prefetch.r (ptr)`

3.3. Scalar Bit Manipulation Extension Intrinsics

In order to access the RISC-V scalar bit manipulation intrinsics, it is necessary to include the header file `riscv_bitmanip.h`.

The functions are only only available if the compiler's `-march` string enables the required ISA extension. (Calling functions for not enabled ISA extensions will lead to compile-time and/or link-time errors.)

Intrinsics operating on XLEN sized value are not available as there is no type defined. If `xlen_t` is added in the future, this can be revisited.

Unsigned types are used as that is the most logical representation for a collection of bits.

Only 32-bit and 64-bit types are supported. In order to increase compatibility, where it is feasible 32-bit intrinsics will be available on RV64. This will sometimes require additional instructions.

No type overloading is supported. This avoids complications from C integer promotion rules and how to handle signed types.

Sign extension of 32-bit values on RV64 is not reflected in the interface.

Table 7. Scalar Bit Manipulation Extension Intrinsics

Prototype	Instruction	Extension	Notes
`unsigned __riscv_clz_32(uint32_t x);`	`clz[w]`	Zbb	
`unsigned __riscv_clz_64(uint64_t x);`	`clz`	Zbb (RV64)	
`unsigned __riscv_ctz_32(uint32_t x);`	`ctz[w]`	Zbb	
`unsigned __riscv_ctz_64(uint64_t x);`	`ctz`	Zbb (RV64)	
`unsigned __riscv_cpop_32(uint32_t x);`	`cpop[w]`	Zbb	
`unsigned __riscv_cpop_64(uint64_t x);`	`cpop`	Zbb (RV64)	

`uint32_t __riscv_orc_b_32(uint32_t x);`	`orc.b`	Zbb	Emulated with `orc.b`+`sext.w` on RV64
`uint64_t __riscv_orc_b_64(uint64_t x);`	`orc.b`	Zbb (RV64)	
`uint32_t __riscv_ror_32(uint32_t x, uint32_t shamt);`	`ror[i][w]`	Zbb, Zbkb	
`uint64_t __riscv_ror_64(uint64_t x, uint32_t shamt);`	`ror[i]`	Zbb, Zbkb (RV64)	
`uint32_t __riscv_rol_32(uint32_t x, uint32_t shamt);`	`rol[w]`/ `rori[w]`	Zbb, Zbkb	
`uint64_t __riscv_rol_64(uint64_t x, uint32_t shamt);`	`rol`/ `rori`	Zbb, Zbkb (RV64)	
`uint32_t __riscv_rev8_32(uint32_t x);`	`rev8`	Zbb, Zbkb	Emulated with `rev8`+`srai` on RV64
`uint64_t __riscv_rev8_64(uint64_t x);`	`rev8`	Zbb, Zbkb (RV64)	
`uint32_t __riscv_brev8_32(uint32_t x);`	`brev8`	Zbkb	Emulated with `brev8`+`sext.w` on RV64
`uint64_t __riscv_brev8_64(uint64_t x);`	`brev8`	Zbkb (RV64)	
`uint32_t __riscv_zip_32(uint32_t x);`	`zip`	Zbkb (RV32)	No emulation for RV64
`uint32_t __riscv_unzip_32(uint32_t x);`	`unzip`	Zbkb (RV32)	No emulation for RV64
`uint32_t __riscv_clmul_32(uint32_t rs1, uint32_t rs2);`	`clmul`	Zbc, Zbkc	Emulated with `clmul`+`sext.w` on RV64
`uint64_t __riscv_clmul_64(uint64_t rs1, uint64_t rs2);`	`clmul`	Zbc, Zbkc (RV64)	
`uint32_t __riscv_clmulh_32(uint32_t rs1, uint32_t rs2);`	`clmulh`	Zbc, Zbkc (RV32)	Emulation on RV64 requires 4-6 instructions
`uint64_t __riscv_clmulh_64(uint64_t rs1, uint64_t rs2);`	`clmulh`	Zbc, Zbkc (RV64)	
`uint32_t __riscv_clmulr_32(uint32_t rs1, uint32_t rs2);`	`clmulr`	Zbc	Emulation on RV64 requires 4-6 instructions
`uint64_t __riscv_clmulr_64(uint64_t rs1, uint64_t rs2);`	`clmulr`	Zbc (RV64)	
`uint32_t __riscv_xperm4_32(uint32_t rs1, uint32_t rs2);`	`xperm4`	Zbkx (RV32)	No emulation for RV64

`uint64_t __riscv_xperm4_64(uint64_t rs1, uint64_t rs2);`	`xperm4`	Zbkx (RV64)	
`uint32_t __riscv_xperm8_32(uint32_t rs1, uint32_t rs2);`	`xperm8`	Zbkx (RV32)	No emulation for RV64
`uint64_t __riscv_xperm8_64(uint64_t rs1, uint64_t rs2);`	`xperm8`	Zbkx (RV64)	

3.4. Scalar Cryptography Extension Intrinsics

In order to access the RISC-V scalar crypto intrinsics, it is necessary to include the header file `riscv_crypto.h`.

The functions are only only available if the compiler's `-march` string enables the required ISA extension. (Calling functions for not enabled ISA extensions will lead to compile-time and/or link-time errors.)

Unsigned types are used as that is the most logical representation for a collection of bits.

Sign extension of 32-bit values on RV64 is not reflected in the interface.

Table 8. Scalar Cryptography Extension Intrinsics

Prototype	Instruction	Extension	Notes
`uint32_t __riscv_aes32dsi(uint32_t rs1, uint32_t rs2, const int bs);`	`aes32dsi`	Zknd (RV32)	`bs`=[0..3]
`uint32_t __riscv_aes32dsmi(uint32_t rs1, uint32_t rs2, const int bs);`	`aes32dsmi`	Zknd (RV32)	`bs`=[0..3]
`uint64_t __riscv_aes64ds(uint64 rs1, uint64_t rs2);`	`aes64ds`	Zknd (RV64)	
`uint64_t __riscv_aes64dsm(uint64 rs1, uint64_t rs2);`	`aes64dsm`	Zknd (RV64)	
`uint64_t __riscv_aes64im(uint64 rs1);`	`aes64im`	Zknd (RV64)	`rnum`=[0..10]
`uint64_t __riscv_aes64ks1i(uint64 rs1, const int rnum);`	`aes64ks1i`	Zknd, Zkne (RV64)	`rnum`=[0..10]
`uint64_t __riscv_aes64ks2(uint64 rs1, uint64_t rs2);`	`aes64ks2`	Zknd, Zkne (RV64)	
`uint32_t __riscv_aes32esi(uint32_t rs1, uint32_t rs2, const int bs);`	`aes32esi`	Zkne (RV32)	`bs`=[0..3]
`uint32_t __riscv_aes32esmi(uint32_t rs1, uint32_t rs2, const int bs);`	`aes32esmi`	Zkne (RV32)	`bs`=[0..3]
`uint64_t __riscv_aes64es(uint64 rs1, uint64_t rs2);`	`aes32es`	Zkne (RV64)	
`uint64_t __riscv_aes64esm(uint64 rs1, uint64_t rs2);`	`aes32esm`	Zkne (RV64)	
`uint32_t __riscv_sha256sig0(uint32_t rs1);`	`sha256sig0`	Zknh	
`uint32_t __riscv_sha256sig1(uint32_t rs1);`	`sha256sig1`	Zknh	

`uint32_t __riscv_sha256sum0(uint32_t rs1);`	`sha256sum0`	Zknh	
`uint32_t __riscv_sha256sum1(uint32_t rs1);`	`sha256sum1`	Zknh	
`uint32_t __riscv_sha512sig0h(uint32_t rs1, uint32_t rs2);`	`sha512sig0h`	Zknh (RV32)	
`uint32_t __riscv_sha512sig0l(uint32_t rs1, uint32_t rs2);`	`sha512sig0l`	Zknh (RV32)	
`uint32_t __riscv_sha512sig1h(uint32_t rs1, uint32_t rs2);`	`sha512sig1h`	Zknh (RV32)	
`uint32_t __riscv_sha512sig1l(uint32_t rs1, uint32_t rs2);`	`sha512sig1l`	Zknh (RV32)	
`uint32_t __riscv_sha512sum0r(uint32_t rs1, uint32_t rs2);`	`sha512sum0r`	Zknh (RV32)	
`uint32_t __riscv_sha512sum1r(uint32_t rs1, uint32_t rs2);`	`sha512sum1r`	Zknh (RV32)	
`uint64_t __riscv_sha512sig0(uint64_t rs1);`	`sha512sig0`	Zknh (RV64)	
`uint64_t __riscv_sha512sig1(uint64_t rs1);`	`sha512sig1`	Zknh (RV64)	
`uint64_t __riscv_sha512sum0(uint64_t rs1);`	`sha512sum0`	Zknh (RV64)	
`uint64_t __riscv_sha512sum1(uint64_t rs1);`	`sha512sum1`	Zknh (RV64)	
`uint32_t __riscv_sm3p0(uint32_t rs1);`	`sm3p0`	Zksh	
`uint32_t __riscv_sm3p1(uint32_t rs1);`	`sm3p1`	Zksh	
`uint32_t __riscv_sm4ed(uint32_t rs1, uint32_t rs2, const int bs);`	`sm4ed`	Zksed	`bs`=[0..3]
`uint32_t __riscv_sm4ks(uint32_t rs1, uint32_t rs2, const int bs);`	`sm4ks`	Zksed	`bs`=[0..3]

3.5. May-Be-Operations Extension Intrinsics

The functions are only available if the compiler's `-march` string enables the required ISA extension. (Calling functions for not enabled ISA extensions will lead to compile-time and/or link-time errors.)

Intrinsics operating on XLEN sized value are not available as there is no type defined. If `xlen_t` is added in the future, this can be revisited.

Unsigned types are used as that is the most logical representation for a collection of bits.

Sign extension of 32-bit values on RV64 is not reflected in the interface.

Table 9. May-Be-Operations Extension Intrinsics

Prototype	Instruction	Extension	Notes
`uint32_t __riscv_mopr_32(uint32_t rs1, const int n);`	`mop.r.[n]`	Zimop	Emulated with `mopr.r.[n]`+`sext.w` on RV64 `n`=[0..31]

`uint64_t __riscv_mopr_64(uint64_t rs1, const int n);`	`mop.r.[n]`	Zimop (RV64)	`n`=[0..31]
`uint32_t __riscv_moprr_32(uint32_t rs1, uint32_t rs2, const int n);`	`mop.rr.[n]`	Zimop	Emulated with `mopr.rr.[n]`+`sext.w` on RV64 `n`=[0..7]
`uint64_t __riscv_moprr_64(uint64_t rs1, uint64_t rs2, const int n);`	`mop.rr.[n]`	Zimop (RV64)	`n`=[0..7]

Chapter 4. Constraints on Operands of Inline Assembly Statements

This section lists operand constraints that can be used with inline assembly statements, including both RISC-V specific and common operand constraints. Operand constraints are case-sensitive.

"Floating-point register" in both the `f` and `cf` rows means "a register suitable for passing a floating-point value", so when using the `Zfinx`, `Zdinx`, or `Zhinxmin` extensions this will allocate an X register. This is done to aid portability of floating-point code.

Table 10. Constraints on Operands of Inline Assembly Statements

Constraint	Description	Note
m	An address that is held in a general-purpose register with offset.	
A	An address that is held in a general-purpose register.	
r	General purpose register	
f	Floating-point register	
i	Immediate integer operand	
I	12-bit signed immediate integer operand	
K	5-bit unsigned immediate integer operand	
J	Zero integer immediate operand	
s	symbol or label reference with a constant offset	
cr	RVC general purpose register (`x8`-`x15`)	
cf	RVC floating-point register (`f8`-`f15` or `x8`-`x15` with `Zfinx`)	
R	Even-odd general purpose register pair	
cR	RVC even-odd general purpose register pair (`x8`-`x14`)	
vr	Vector register	
vd	Vector register, excluding v0	
vm	Vector register, only v0	

The `R` constraint should print as the even register in the pair, as this matches how the `amocas.q` instruction (on RV64) or the `amocas.d` and `Zdinx` instructions (on RV32) expect to parse their pair register operands. However, both registers in the pair should be considered to be live or clobbered together.

Immediate value must be a compile-time constant.

The `c*` constraints are designed to be extensible to more kinds of RVC-compatible register constraints in the future.

4.1. The Difference Between m and A Constraints

The difference between m and A is whether the operand can have an offset; some instructions in RISC-V do not allow an offset for the address operand, such as atomic or vector load/store instructions.

The following example demonstrates the difference; it is trying to load value from foo[10] and using m and A to pass that address.

```
int *foo;
void bar() {
 int x;
 __asm__ volatile ("lw %0, %1" : "=r"(x) : "m" (foo[10]));
 __asm__ volatile ("lw %0, %1" : "=r"(x) : "A" (foo[10]));
}
```

Then we compile with GCC with -O option:

```
$ riscv64-unknown-elf-gcc x.c -o - -O -S
...
bar:
       lui    a5,%hi(foo)
       ld     a5,%lo(foo)(a5)
 #APP
# 4 "x.c" 1
       lw a4, 40(a5)
# 0 "" 2
 #NO_APP
       addi   a5,a5,40
 #APP
# 5 "x.c" 1
       lw a5, 0(a5)
# 0 "" 2
 #NO_APP
       ret
```

The compiler uses an immediate offset of 40 for the m constraint, but for the A constraint uses an extra addi instruction instead.

4.2. Operand Modifiers

This section lists operand modifiers that can be used with inline assembly statements, including both RISC-V specific and common operand modifiers.

Table 11. Operand Modifiers

Modifiers	Description	Note

z	Print `zero` (`x0`) register for immediate 0, typically used with constraints `J`	
i	Print `i` if corresponding operand is immediate.	
N	Print register encoding as integer (0-31).	

Chapter 5. Function Multi-version

Function multi-versioning (FMV) allows selecting the appropriate function based on the runtime environment. The binary may contain multiple versions of the function, with the compiler generating all supported versions and selecting the appropriate one at runtime.

This feature is activated by the `target_version/target_clones` function attributes.

5.1. Extension Bitmask

The Extension Bitmask is used to probe whether features are enabled during runtime. This is achieved through three bitmask structures: `riscv_feature_bits for standard extensions andriscv_cpu_model` for the CPU model. Additionally, `__init_riscv_feature_bits` is used to update the contents of these structures according to the system configuration.

The bitmask structures use the following definitions:

```
struct {
    unsigned length;
    unsigned long long features[];
} __riscv_feature_bits;

struct {
  unsigned mvendorid;
  unsigned long long marchid;
  unsigned long long mimpid;
} __riscv_cpu_model;
```

- `length`: Represents the number of elements in the features array.
- `features`: An `unsigned long long` array where each bit indicates 1 for a specific extension enabled by the system or 0 for the extension's status unknown in system.
- `mvendorid`: Indicates the value of `mvendorid` CSR.
- `marchid`: Indicates the value of `marchid` CSR.
- `mimpid`: Indicates the value of `mimpid` CSR.

To initiate these structures based on the system's extension status, the following function is provided:

```
void __init_riscv_feature_bits(void *);
```

The `init_riscv_feature_bits function updates length, mvendorid, marchid, mimpid and the features inriscv_feature_bits` according to the enabled extensions in the system.

The `__init_riscv_feature_bits` function accepts an argument of type `void *`. This argument allows the platform to provide pre-computed data and access it without additional effort. For example,

Linux could pass the vDSO object to avoid an extra system call.

All `length` fields should be initialized to zero if `__init_riscv_feature_bits` fails to detect the features.

> To detect failure in the `init_riscv_feature_bits function, it is recommended to check thatriscv_feature_bits.length` is non-zero.

Each queryable extension must have an associated `groupid` and `bitmask` that indicates its position within the features array.

> For example, the zba extension is represented by `groupid`: 0 and `bitmask`: `1ULL << 27`. Users can check if the zba extension is enabled using: `__riscv_feature_bits.features[0] & (1ULL << 27)`.

5.1.1. Extension Bitmask Definitions

> The single-letter extension bitmask follows the `misa` bit position inside `__riscv_feature_bits.features[0]`.

extension	groupid	bit position
a	0	0
b	0	1
c	0	2
d	0	3
e	0	4
f	0	5
h	0	7
i	0	8
m	0	12
q	0	16
v	0	21
zacas	0	26
zba	0	27
zbb	0	28
zbc	0	29
zbkb	0	30
zbkc	0	31
zbkx	0	32
zbs	0	33

zfa	0	34
zfh	0	35
zfhmin	0	36
zicboz	0	37
zicond	0	38
zihintntl	0	39
zihintpause	0	40
zknd	0	41
zkne	0	42
zknh	0	43
zksed	0	44
zksh	0	45
zkt	0	46
ztso	0	47
zvbb	0	48
zvbc	0	49
zvfh	0	50
zvfhmin	0	51
zvkb	0	52
zvkg	0	53
zvkned	0	54
zvknha	0	55
zvknhb	0	56
zvksed	0	57
zvksh	0	58
zvkt	0	59
zve32x	0	60
zve32f	0	61
zve64x	0	62
zve64f	0	63
zve64d	1	0
zimop	1	1
zca	1	2
zcb	1	3

zcd	1	4
zcf	1	5
zcmop	1	6
zawrs	1	7
zilsd	1	8
zclsd	1	9
zcmp	1	10
zifencei	1	11
zmmul	1	12

5.2. Version Selection

The process of selecting the appropriate function version during function multi-versioning follows these guidelines:

1. The implementation of the selection algorithm is implementation-specific.
2. Once a version is selected, it remains in use for the entire duration of the process.
3. Only versions whose required features are all available in the runtime environment are eligible for selection.

The version selection process applies the following rules in order:

1. Among the eligible versions, select the one with the highest priority.
2. If multiple versions have equal priority, select one based on an implementation-defined heuristic.
3. If no other suitable versions are found, fall back to the "default" version.